Python DSA Interview Roadmap

Direct LeetCode links for ■6+ LPA and top MNC software roles

Topic explanations • Python-focused patterns • 3-hour daily plan

How to use this roadmap

Try each problem for 20–30 minutes before checking hints. Explain your approach, code it in Python, test edge cases, and state time/space complexity. Re-solve difficult problems after 1, 3, and 7 days.

Suggested 3-hour daily routine

Time	Task
45 min	Learn the topic / watch one explanation
75 min	Solve 2–3 problems in Python
30 min	Review mistakes and complexity
30 min	Re-solve previous problems without notes

Primary reference resources

LeetCode 75 official study plan · LeetCode Top Interview 150 · NeetCode roadmap (Python solutions and video explanations)

This is a focused, curated plan—not a guarantee of employment or salary. DSA is one part of interviews; also prepare Python fundamentals, SQL, APIs, Git, projects, and communication.

PHASE 1 — FOUNDATIONS (WEEKS 1–2)

1. Big-O, Arrays & Strings

What it means: Learn how time and memory scale with input size. Practise one-pass scans, indexing, in-place updates, and common string/array edge cases.

Learn to: Explain O(1), O(log n), O(n), O(n log n), O(n^2). For every solution, state time and auxiliary-space complexity.

Two Sum [Easy]

Focus: Hash map: find the complement in O(n).

Open direct LeetCode problem ■

Best Time to Buy and Sell Stock [Easy]

Focus: Track minimum price so far and best profit.

Open direct LeetCode problem ■

Contains Duplicate [Easy]

Focus: Use a set for fast membership checks.

Open direct LeetCode problem ■

Product of Array Except Self [Medium]

Focus: Combine prefix and suffix products.

Open direct LeetCode problem ■

Maximum Subarray [Medium]

Focus: Kadane's algorithm tracks the best ending here.

Open direct LeetCode problem ■

Move Zeroes [Easy]

Focus: Use a write pointer and preserve order.

Open direct LeetCode problem ■

Valid Anagram [Easy]

Focus: Compare character frequencies.

Open direct LeetCode problem ■

Longest Common Prefix [Easy]

Focus: Stop at the first mismatch across strings.

Open direct LeetCode problem ■

2. Hash Maps, Sets & Prefix Sums

What it means: Use dictionaries and sets for fast lookups/counting. Prefix sums answer cumulative range/subarray questions efficiently.

Learn to: A hash map often replaces nested loops. Prefix sums work with negative values; a basic sliding window often needs non-negative values.

Group Anagrams [Medium]

Focus: Group by sorted letters or a frequency signature.

Open direct LeetCode problem ■

Top K Frequent Elements [Medium]

Focus: Count frequencies, then use a heap or buckets.

Open direct LeetCode problem ■

Subarray Sum Equals K [Medium]

Focus: Prefix-sum frequency map.

Open direct LeetCode problem ■

Find Pivot Index [Easy]

Focus: Compare left sum with right sum.

Open direct LeetCode problem ■

Longest Consecutive Sequence [Medium]

Focus: Use a set and start only at sequence beginnings.

Open direct LeetCode problem ■

PHASE 2 — HIGH-FREQUENCY PATTERNS (WEEKS 3–4)

3. Two Pointers & Sliding Window

What it means: Two pointers coordinate positions; a sliding window maintains a contiguous range while adding/removing elements to satisfy a condition.

Learn to: Use opposite-end pointers on sorted data or palindromes. Use fixed windows for fixed-size ranges and variable windows for constraints.

Valid Palindrome [Easy]

Focus: Move inward while skipping non-alphanumeric characters.

Open direct LeetCode problem ■

Two Sum II - Input Array Is Sorted [Medium]

Focus: Move left/right according to the sum.

Open direct LeetCode problem ■

3Sum [Medium]

Focus: Sort, then sweep with two pointers.

Open direct LeetCode problem ■

Container With Most Water [Medium]

Focus: Move the pointer at the shorter boundary.

Open direct LeetCode problem ■

Longest Substring Without Repeating Characters [Medium]

Focus: Variable window plus last-seen positions.

Open direct LeetCode problem ■

Minimum Size Subarray Sum [Medium]

Focus: Shrink a positive-number window when the target is met.

Open direct LeetCode problem ■

4. Binary Search & Intervals

What it means: Binary search halves a sorted or monotonic search space. Interval questions use sorting to detect overlaps and choose compatible ranges.

Learn to: Keep boundary conditions consistent; test duplicates and first/last positions. For answer-space search, define a feasibility check.

Binary Search [Easy]

Focus: Classic O(log n) search.

Open direct LeetCode problem ■

Search Insert Position [Easy]

Focus: Lower-bound binary search.

Open direct LeetCode problem ■

Find Minimum in Rotated Sorted Array [Medium]

Focus: Compare the middle with the right boundary.

Open direct LeetCode problem ■

Search in Rotated Sorted Array [Medium]

Focus: Identify which half is sorted.

Open direct LeetCode problem ■

Koko Eating Bananas [Medium]

Focus: Binary search the answer with a feasibility function.

Open direct LeetCode problem ■

Merge Intervals [Medium]

Focus: Sort by start and merge overlaps.

Open direct LeetCode problem ■

Insert Interval [Medium]

Focus: Handle before, overlap, and after intervals.

Open direct LeetCode problem ■

PHASE 3 — DATA STRUCTURES (WEEKS 5–6)

5. Stacks, Queues & Linked Lists

What it means: Stacks are last-in-first-out; queues are first-in-first-out. Linked lists test pointer changes and edge-case handling.

Learn to: For stacks, ask what must wait until a later element. For linked lists, draw previous/current/next pointers and handle empty or one-node lists.

Valid Parentheses [Easy]

Focus: Stack of opening brackets.

Open direct LeetCode problem ■

Min Stack [Medium]

Focus: Maintain the minimum in O(1).

Open direct LeetCode problem ■

Daily Temperatures [Medium]

Focus: Monotonic stack resolves waiting days.

Open direct LeetCode problem ■

Evaluate Reverse Polish Notation [Medium]

Focus: Evaluate operands using a stack.

Open direct LeetCode problem ■

Reverse Linked List [Easy]

Focus: Update previous/current/next pointers.

Open direct LeetCode problem ■

Merge Two Sorted Lists [Easy]

Focus: Dummy head plus two pointers.

Open direct LeetCode problem ■

Linked List Cycle [Easy]

Focus: Floyd's fast/slow pointers.

Open direct LeetCode problem ■

Remove Nth Node From End of List [Medium]

Focus: Two pointers separated by n nodes.

Open direct LeetCode problem ■

6. Trees & Binary Search Trees

What it means: DFS explores depth and paths; BFS processes levels. BST ordering enables efficient search and ordered traversal.

Learn to: Know recursive base cases, returned subtree values, queue-based level order, and inorder traversal for BSTs.

Maximum Depth of Binary Tree [Easy]

Focus: DFS recursion.

Open direct LeetCode problem ■

Invert Binary Tree [Easy]

Focus: Swap left and right children recursively.

Open direct LeetCode problem ■

Binary Tree Level Order Traversal [Medium]

Focus: BFS with a queue.

Open direct LeetCode problem ■

Diameter of Binary Tree [Easy]

Focus: Postorder height plus best diameter.

Open direct LeetCode problem ■

Validate Binary Search Tree [Medium]

Focus: Propagate strict lower/upper bounds.

Open direct LeetCode problem ■

Lowest Common Ancestor of a Binary Tree [Medium]

Focus: Combine results from subtrees.

Open direct LeetCode problem ■

PHASE 4 — INTERVIEW ALGORITHMS (WEEKS 7–8)

7. Heaps, Greedy & Backtracking

What it means: Heaps select min/max items efficiently. Greedy uses locally optimal choices supported by an invariant. Backtracking explores choices and undoes them.

Learn to: In Python, practise heapq. For backtracking, define choices, constraints, base case, and undo step.

Kth Largest Element in an Array [Medium]

Focus: Heap or quickselect.

Open direct LeetCode problem ■

Last Stone Weight [Easy]

Focus: Use a max-heap via negative values in Python.

Open direct LeetCode problem ■

Task Scheduler [Medium]

Focus: Frequency counts and greedy scheduling.

Open direct LeetCode problem ■

Subsets [Medium]

Focus: Backtracking or bit masks.

Open direct LeetCode problem ■

Combination Sum [Medium]

Focus: Choose, recurse, then backtrack.

Open direct LeetCode problem ■

Permutations [Medium]

Focus: Track choices already used.

Open direct LeetCode problem ■

8. Graphs

What it means: Model entities as vertices and relationships as edges. BFS finds shortest paths in unweighted graphs; DFS explores components and paths.

Learn to: Use adjacency lists, visited sets, and collections.deque. Mark nodes at the right time to prevent repeated work.

Number of Islands [Medium]

Focus: Grid DFS/BFS and visited cells.

Open direct LeetCode problem ■

Flood Fill [Easy]

Focus: Explore connected neighboring cells.

Open direct LeetCode problem ■

Clone Graph [Medium]

Focus: DFS/BFS with an old-to-new map.

Open direct LeetCode problem ■

Course Schedule [Medium]

Focus: Cycle detection or topological ordering.

Open direct LeetCode problem ■

Rotting Oranges [Medium]

Focus: Multi-source BFS.

Open direct LeetCode problem ■

Pacific Atlantic Water Flow [Medium]

Focus: Reverse reachability from ocean borders.

Open direct LeetCode problem ■

9. Dynamic Programming

What it means: DP stores results for overlapping subproblems. Define a state, recurrence, base cases, and evaluation order before coding.

Learn to: Start with recursion and memoization, then tabulation. Be able to explain exactly what dp[i] represents.

Climbing Stairs [Easy]

Focus: One-dimensional recurrence.

Open direct LeetCode problem ■

House Robber [Medium]

Focus: Choose this house or skip it.

Open direct LeetCode problem ■

Coin Change [Medium]

Focus: Minimum coins for every amount.

Open direct LeetCode problem ■

Unique Paths [Medium]

Focus: Grid DP.

Open direct LeetCode problem ■

Longest Increasing Subsequence [Medium]

Focus: DP $O(n^2)$, then optimize to $O(n \log n)$.

Open direct LeetCode problem ■

Word Break [Medium]

Focus: Prefix DP with dictionary lookup.

Open direct LeetCode problem ■

10. Bit Manipulation & Interview Implementation

What it means: Use bit operations for compact state logic; write readable Python and test edge cases. Interviewers also expect you to explain trade-offs, not just submit code.

Learn to: Practise dict/set/list operations, sorted(key=...), heapq, Counter, defaultdict, deque, and clear tests.

Single Number [Easy]

Focus: XOR cancels matching pairs.

Open direct LeetCode problem ■

Number of 1 Bits [Easy]

Focus: Count set bits.

Open direct LeetCode problem ■

Majority Element [Easy]

Focus: Boyer–Moore voting or frequency counting.

Open direct LeetCode problem ■

Find the Duplicate Number [Medium]

Focus: Cycle detection or value-domain binary search.

Open direct LeetCode problem ■

LRU Cache [Medium]

Focus: Hash map plus doubly linked list.

Open direct LeetCode problem ■

Interview readiness checklist

- ■ Solve core Easy problems without hints and explain the solution.
- ■ Solve many Medium problems using a known pattern and discuss trade-offs.
- ■ State time and space complexity and test edge cases.
- ■ Explain one or two projects end-to-end, including design choices and bugs fixed.
- ■ Revise Python, SQL, REST APIs, Git, and role-specific backend/AI skills alongside DSA.
- ■ Complete timed practice and mock interviews before applying to target roles.

Sources and selection method

1. LeetCode 75 — structured interview preparation. 2. LeetCode Top Interview 150 — broader classic interview coverage. 3. NeetCode roadmap — topic order, problem groups, Python solutions and videos. 4. NeetCode interview preparation guide — pattern learning and re-solving. Each problem link in this PDF opens its direct LeetCode page.

Total curated direct-link problems: 63. This is a focused starting list, not an exhaustive list of every possible interview question.